RiceSeedCNN

May 1, 2026

0.0.1 Basmati Rice Classification (Custom CNN)

[1]:
```
import os
import torch
import torch.nn.functional as F
import torch.nn as nn
from torchvision import datasets, transforms
from torch.utils.data import DataLoader
import random
import numpy as np
from sklearn.metrics import classification_report, confusion_matrix
import matplotlib.pyplot as plt
import seaborn as sns
```

[2]:
```
# Squeeze and Excite block implementation
class SEBlock(nn.Module):
    def __init__(self, channels, reduction=16):
        super(SEBlock, self).__init__()
        self.avg_pool = nn.AdaptiveAvgPool2d(1)
        self.fc = nn.Sequential(
            nn.Linear(channels, channels // reduction, bias=False),
            nn.ReLU(inplace=True),
            nn.Linear(channels // reduction, channels, bias=False),
            nn.Sigmoid()
        )

    def forward(self, x):
        b, c, hgt, wdth = x.size()
        y = self.avg_pool(x).view(b, c)
        y = self.fc(y).view(b, c, 1, 1)
        return x * y.expand_as(x)

# Combine convolutional layers with a Squeeze-and-Excite block for better␣
 ↪feature learning and channel-wise attention.
class ConvSEBlock(nn.Module):
    def __init__(self, in_channels, out_channels, stride=1):
        super(ConvSEBlock, self).__init__()
```

```
        self.conv1 = nn.Conv2d(in_channels, out_channels, kernel_size=3,␣
 ↪stride=stride, padding=1, bias=False)
        self.bn1 = nn.BatchNorm2d(out_channels, momentum=0.05)
        self.conv2 = nn.Conv2d(out_channels, out_channels, kernel_size=3,␣
 ↪stride=1, padding=1, bias=False)
        self.bn2 = nn.BatchNorm2d(out_channels, momentum=0.05)

        self.se = SEBlock(out_channels)

    def forward(self, x):
        out = F.relu(self.bn1(self.conv1(x)))
        out = self.bn2(self.conv2(out))
        out = self.se(out)
        out = F.relu(out)
        return out

class SeedCNN(nn.Module):
    def __init__(self, num_classes):
        super(SeedCNN, self).__init__()
        self.conv1 = nn.Conv2d(3, 32, kernel_size=3, stride=1, padding=1,␣
 ↪bias=False)
        self.bn1 = nn.BatchNorm2d(32)

        self.layer1 = ConvSEBlock(32, 32, stride=1)
        self.layer2 = ConvSEBlock(32, 64, stride=2)
        self.layer3 = ConvSEBlock(64, 128, stride=2)
        self.layer4 = ConvSEBlock(128, 256, stride=2)
        self.dropout = nn.Dropout(p=0.4)
        self.linear = nn.Linear(256, num_classes)

    def forward(self, x):
        out = F.relu(self.bn1(self.conv1(x)))
        out = self.layer1(out)
        out = self.layer2(out)
        out = self.layer3(out)
        out = self.layer4(out)
        out = F.adaptive_avg_pool2d(out, 1)
        out = out.view(out.size(0), -1)
        out = self.dropout(out)
        out = self.linear(out)
        return out
```

[3]:
```
def get_data_loaders(data_dir, batch_size=16):
    data_transforms = {
        'train': transforms.Compose([
            transforms.RandomResizedCrop(224, scale=(0.8, 1.0)),
            transforms.RandomHorizontalFlip(),
```

```
            transforms.RandomVerticalFlip(),
            transforms.RandomRotation(30),
            transforms.ColorJitter(brightness=0.2, contrast=0.2),
            transforms.ToTensor(),
            transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])
        ]),
        'val': transforms.Compose([
            transforms.Resize((224, 224)),
            transforms.ToTensor(),
            transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])
        ]),
    }

    # getting images
    image_datasets = {
        'train': datasets.ImageFolder(os.path.join(data_dir, "train"),␣
↪data_transforms['train']),
        'val': datasets.ImageFolder(os.path.join(data_dir, "test_val"),␣
↪data_transforms['val']),
        'test': datasets.ImageFolder(os.path.join(data_dir, "test"),␣
↪data_transforms['val'])
    }

    dataloaders = {
        x: DataLoader(
            image_datasets[x],
            batch_size=batch_size,
            shuffle=(x == 'train'),
            num_workers=2,
            pin_memory=True
        )
        for x in ['train', 'val', 'test']
    }

    return dataloaders, image_datasets['train'].class_to_idx
```

[4]:
```
def visualize_seed_varieties(dataloader, class_map, num_images=6):
    print("\nVisualizing a batch of seed varieties...")
    # Get a batch of training data
    inputs, classes = next(iter(dataloader))

    # Reverse the normalization so images look normal
    def imshow(inp, title=None):
        inp = inp.numpy().transpose((1, 2, 0))
        mean = np.array([0.485, 0.456, 0.406])
        std = np.array([0.229, 0.224, 0.225])
        inp = std * inp + mean
```

```
        inp = np.clip(inp, 0, 1)
        plt.imshow(inp)
        if title is not None:
            plt.title(title, fontsize=10)
        plt.axis('off')

    # Reverse class_map to get names from indices
    idx_to_class = {v: k for k, v in class_map.items()}

    # Plot the images
    plt.figure(figsize=(15, 6))
    for i in range(min(num_images, inputs.size(0))):
        plt.subplot(2, int(np.ceil(num_images/2)), i + 1)
        imshow(inputs[i], title=idx_to_class[classes[i].item()])

    plt.suptitle('Sample Rice Seed Varieties (Preprocessed)', fontsize=16)
    plt.tight_layout()
    plt.savefig('seed_varieties_sample.png', dpi=150)
    print("Saved sample visualization as 'seed_varieties_sample.png'.")
    plt.show()
```

[5]:
```
def train_and_validate_model(model, dataloaders, criterion, optimizer, device,␣
↪num_epochs=20, patience=5):
    history = {'train_loss': [],'train_acc': [], 'val_loss': [], 'val_acc': []}
    accumulation_step = 4

    best_val_loss = float('inf')
    epochs_no_improve = 0

    for epoch in range(num_epochs):
        model.train()
        train_loss = 0.0
        train_correct = 0

        for i, (inputs, labels) in enumerate(dataloaders['train']):
            inputs, labels = inputs.to(device), labels.to(device)
            outputs = model(inputs)
            loss = criterion(outputs, labels)

            train_loss += loss.item() * inputs.size(0)

            loss_accumulated = loss / accumulation_step
            loss_accumulated.backward()

            if (i + 1) % accumulation_step == 0 or (i + 1) ==␣
↪len(dataloaders['train']):
                optimizer.step()
```

```
                optimizer.zero_grad()

            _, preds = torch.max(outputs, 1)
            train_correct += torch.sum(preds == labels.data)

        avg_train_loss = train_loss / len(dataloaders['train'].dataset)
        train_acc = train_correct.double() / len(dataloaders['train'].dataset)

        # Validation
        model.eval()
        val_loss = 0.0
        correct = 0

        with torch.no_grad():
            for inputs, labels in dataloaders['val']:
                inputs, labels = inputs.to(device), labels.to(device)
                outputs = model(inputs)
                v_loss = criterion(outputs, labels)
                val_loss += v_loss.item() * inputs.size(0)
                _, preds = torch.max(outputs, 1)
                correct += torch.sum(preds == labels.data)

        avg_val_loss = val_loss / len(dataloaders['val'].dataset)
        val_acc = correct.double() / len(dataloaders['val'].dataset)

        history['train_loss'].append(avg_train_loss)
        history['train_acc'].append(train_acc.item())
        history['val_loss'].append(avg_val_loss)
        history['val_acc'].append(val_acc.item())

        print(f"Epoch {epoch+1}/{num_epochs} | Train Acc: {train_acc:.4f} |␣
↪Train Loss: {avg_train_loss:.4f} | Val Acc: {val_acc:.4f} | Val Loss:␣
↪{avg_val_loss:.4f}")

        # Early Stopping & Model Saving Logic
        if avg_val_loss < best_val_loss:
            best_val_loss = avg_val_loss
            epochs_no_improve = 0
            torch.save(model.state_dict(), 'best_model_cnn.pth')
            print(f"  -> Saved new best model! (Val Loss: {best_val_loss:.4f})")
        else:
            epochs_no_improve += 1
            print(f"  -> No improvement for {epochs_no_improve} epoch(s).")

        if epochs_no_improve >= patience:
            print(f"\n[!] Early stopping triggered! Validation loss hasn't␣
↪improved for {patience} epochs.")
```

```
            print("[!] Halting training to prevent overfitting.\n")
            break

    return history
```

[6]:
```
def plot_training_history(history):
    print("\nGenerating training history plots...")
    epochs = range(1, len(history['train_loss']) + 1)

    plt.figure(figsize=(14, 5))

    # Accuracy plot
    plt.subplot(1, 2, 1)
    plt.plot(epochs, history['train_acc'], 'b-', label='Training Accuracy')
    plt.plot(epochs, history['val_acc'], 'r-', label='Validation Accuracy')
    plt.title('Training and Validation Accuracy')
    plt.xlabel('Epochs')
    plt.ylabel('Accuracy')
    plt.legend()
    plt.grid(True, linestyle='--', alpha=0.7)

    # Loss Plot
    plt.subplot(1, 2, 2)
    plt.plot(epochs, history['train_loss'], 'b-', label='Training Loss')
    plt.plot(epochs, history['val_loss'], 'r-', label='Validation Loss')
    plt.title('Training and Validation Loss')
    plt.xlabel('Epochs')
    plt.ylabel('Loss')
    plt.legend()
    plt.grid(True, linestyle='--', alpha=0.7)

    plt.tight_layout()
    plt.savefig('training_history_plots_cnn.png', dpi=300)
    print("Saved training history plots as 'training_history_plots_cnn.png'.")
    plt.show()
```

[7]:
```
def get_predictions(model, dataloader,class_names, device):
    model.load_state_dict(torch.load('best_model_cnn.pth', weights_only=True))
    model.eval()
    all_preds = []
    all_labels = []

    with torch.no_grad():
        for inputs, labels in dataloader:
            inputs = inputs.to(device)
            outputs = model(inputs)
            _, preds = torch.max(outputs, 1)
```

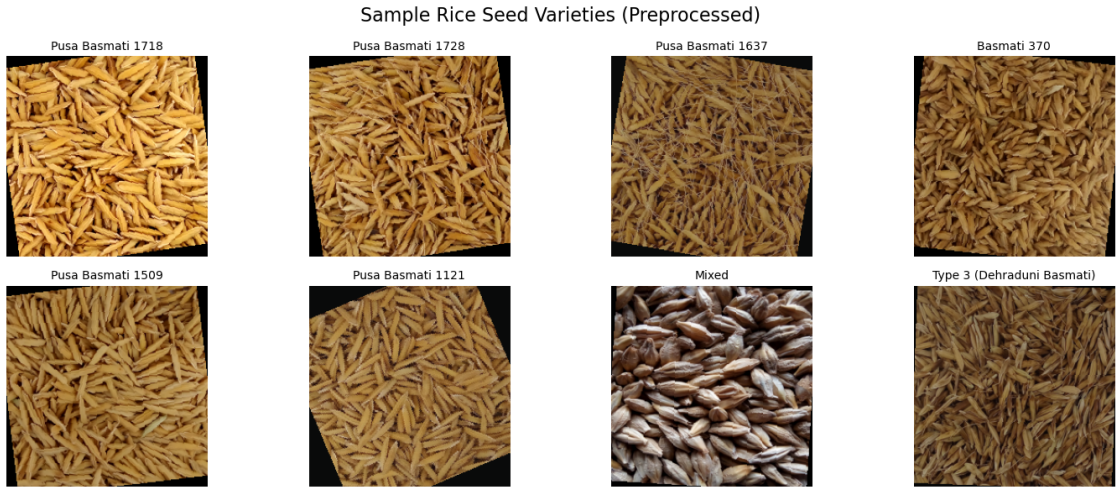
```
            all_preds.extend(preds.cpu().numpy())
            all_labels.extend(labels.numpy())

    #return np.array(all_labels), np.array(all_preds)
    y_true = np.array(all_labels)
    y_pred = np.array(all_preds)
    # Print Classification Report
    print("\nClassification Report:\n")
    print(classification_report(y_true, y_pred, target_names=class_names))

    # Plot Confusion Matrix
    cm = confusion_matrix(y_true, y_pred)

    plt.figure(figsize=(12, 10))
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
                xticklabels=class_names,
                yticklabels=class_names,
                cbar=False,
                linewidths=0.5,
                linecolor='gray')

    plt.title('Basmati Rice Seed Classification - Confusion Matrix (Custom␣
↪CNN)', fontsize=16, pad=20)
    plt.ylabel('True Class (Actual Seed)', fontsize=14, labelpad=15)
    plt.xlabel('Predicted Class (Model Guess)', fontsize=14, labelpad=15)
    plt.xticks(rotation=45, ha='right', fontsize=11)
    plt.yticks(rotation=0, fontsize=11)

    plt.tight_layout()
    plt.savefig('confusion_matrix_heatmap_cnn.png', dpi=300,␣
↪bbox_inches='tight')
    print("\nSaved confusion matrix heatmap as 'confusion_matrix_heatmap_cnn.
↪png'.")
    plt.show()
```

[8]:
```
if __name__ == '__main__':

    #Setting seeds for reproducibility
    seed = 5
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    torch.cuda.manual_seed(seed)
    torch.backends.cudnn.deterministic = True
    torch.backends.cudnn.benchmark = False
```

```
    # Input dataset path
    print("=== Basmati Rice Seed Classification ===")
    DATA_PATH = input(r"Please enter the path to the dataset directory (e.g., D:
↪\Project\iRSVPred_data): ").strip()

    # Check if folder exists
    if not os.path.exists(DATA_PATH):
        raise FileNotFoundError(f"The directory '{DATA_PATH}' does not exist.␣
↪Please check the path and try running the script again.")

    DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    print(f"\nUsing device: {DEVICE}")

    # Get Pretty Names
    folder_to_name_map = {
        "1121": "Pusa Basmati 1121",
        "1509": "Pusa Basmati 1509",
        "1637": "Pusa Basmati 1637",
        "1728": "Pusa Basmati 1728",
        "1718": "Pusa Basmati 1718",
        "BAS_370": "Basmati 370",
        "CSR_30": "Basmati CSR 30",
        "DHBT_3": "Type 3 (Dehraduni Basmati)",
        "PB_6": "Pusa Basmati 6",
        "PB1": "Pusa Basmati 1",
        "Unknown": "Mixed"
    }
    # Initialize
    loaders, class_map = get_data_loaders(DATA_PATH)
    print(f"Loaded {len(class_map)} classes: {class_map}\n")
    pytorch_folders_sorted = list(class_map.keys())
    try:
        pretty_class_names = [folder_to_name_map[folder] for folder in␣
↪pytorch_folders_sorted]
    except KeyError as e:
        print(f"\n[ERROR] PyTorch found a folder named {e} but it's not in your␣
↪dictionary!")
        print(f"Folders PyTorch found: {pytorch_folders_sorted}")

    #Visualize Data Preprocessing
    visualize_seed_varieties(loaders['train'], dict(zip(pretty_class_names,␣
↪class_map.values())), num_images=8)

    model = SeedCNN(num_classes=len(class_map)).to(DEVICE)

    # Standard Optimizer
```

```
    optimizer = torch.optim.Adam(model.parameters(), lr=0.0001,␣
↪weight_decay=1e-3)
    criterion = torch.nn.CrossEntropyLoss()

    # Start Training
    results_history = train_and_validate_model(model, loaders, criterion,␣
↪optimizer, DEVICE, num_epochs=30, patience=7)
    # Plot Training History
    plot_training_history(results_history)

    # Evaluate Test set
    print("\nEvaluating best model on Test Set...")
    get_predictions(model, loaders['test'], pretty_class_names, DEVICE)
```

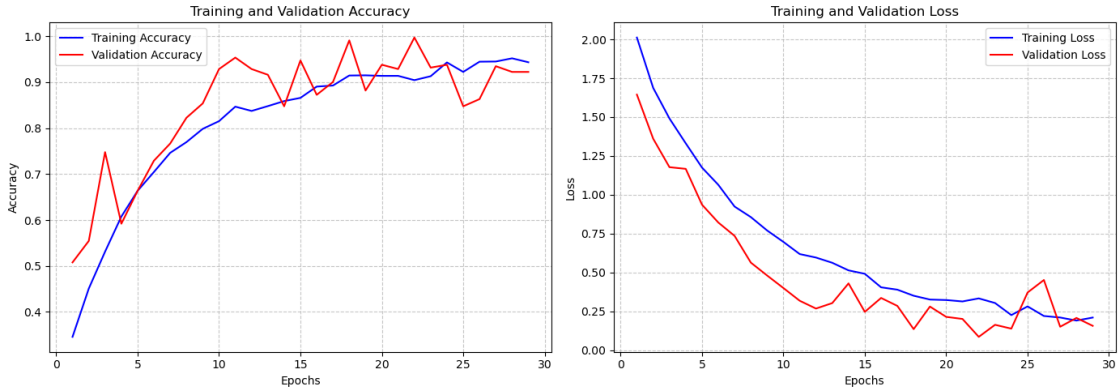
```
=== Basmati Rice Seed Classification ===

Please enter the path to the dataset directory (e.g., D:\Project\iRSVPred_data):
D:\Project\iRSVPred_data


Using device: cuda
Loaded 11 classes: {'1121': 0, '1509': 1, '1637': 2, '1718': 3, '1728': 4,
'BAS_370': 5, 'CSR_30': 6, 'DHBT_3': 7, 'PB1': 8, 'PB_6': 9, 'Unknown': 10}


Visualizing a batch of seed varieties…
Saved sample visualization as 'seed_varieties_sample.png'.
```




```
Epoch 1/30 | Train Acc: 0.3458 | Train Loss: 2.0117 | Val Acc: 0.5078 | Val
Loss: 1.6447
  -> Saved new best model! (Val Loss: 1.6447)
Epoch 2/30 | Train Acc: 0.4509 | Train Loss: 1.6880 | Val Acc: 0.5545 | Val
```

```
Loss: 1.3611
  -> Saved new best model! (Val Loss: 1.3611)
Epoch 3/30 | Train Acc: 0.5312 | Train Loss: 1.4912 | Val Acc: 0.7477 | Val
Loss: 1.1779
  -> Saved new best model! (Val Loss: 1.1779)
Epoch 4/30 | Train Acc: 0.6075 | Train Loss: 1.3314 | Val Acc: 0.5919 | Val
Loss: 1.1670
  -> Saved new best model! (Val Loss: 1.1670)
Epoch 5/30 | Train Acc: 0.6636 | Train Loss: 1.1753 | Val Acc: 0.6636 | Val
Loss: 0.9366
  -> Saved new best model! (Val Loss: 0.9366)
Epoch 6/30 | Train Acc: 0.7044 | Train Loss: 1.0645 | Val Acc: 0.7290 | Val
Loss: 0.8220
  -> Saved new best model! (Val Loss: 0.8220)
Epoch 7/30 | Train Acc: 0.7461 | Train Loss: 0.9246 | Val Acc: 0.7664 | Val
Loss: 0.7361
  -> Saved new best model! (Val Loss: 0.7361)
Epoch 8/30 | Train Acc: 0.7695 | Train Loss: 0.8568 | Val Acc: 0.8224 | Val
Loss: 0.5639
  -> Saved new best model! (Val Loss: 0.5639)
Epoch 9/30 | Train Acc: 0.7983 | Train Loss: 0.7707 | Val Acc: 0.8536 | Val
Loss: 0.4810
  -> Saved new best model! (Val Loss: 0.4810)
Epoch 10/30 | Train Acc: 0.8150 | Train Loss: 0.6971 | Val Acc: 0.9283 | Val
Loss: 0.4005
  -> Saved new best model! (Val Loss: 0.4005)
Epoch 11/30 | Train Acc: 0.8466 | Train Loss: 0.6191 | Val Acc: 0.9533 | Val
Loss: 0.3187
  -> Saved new best model! (Val Loss: 0.3187)
Epoch 12/30 | Train Acc: 0.8372 | Train Loss: 0.5965 | Val Acc: 0.9283 | Val
Loss: 0.2684
  -> Saved new best model! (Val Loss: 0.2684)
Epoch 13/30 | Train Acc: 0.8477 | Train Loss: 0.5629 | Val Acc: 0.9159 | Val
Loss: 0.3035
  -> No improvement for 1 epoch(s).
Epoch 14/30 | Train Acc: 0.8586 | Train Loss: 0.5140 | Val Acc: 0.8474 | Val
Loss: 0.4302
  -> No improvement for 2 epoch(s).
Epoch 15/30 | Train Acc: 0.8657 | Train Loss: 0.4919 | Val Acc: 0.9470 | Val
Loss: 0.2473
  -> Saved new best model! (Val Loss: 0.2473)
Epoch 16/30 | Train Acc: 0.8902 | Train Loss: 0.4050 | Val Acc: 0.8723 | Val
Loss: 0.3366
  -> No improvement for 1 epoch(s).
Epoch 17/30 | Train Acc: 0.8925 | Train Loss: 0.3899 | Val Acc: 0.9003 | Val
Loss: 0.2854
  -> No improvement for 2 epoch(s).
Epoch 18/30 | Train Acc: 0.9143 | Train Loss: 0.3509 | Val Acc: 0.9907 | Val
```

```
Loss: 0.1361
  -> Saved new best model! (Val Loss: 0.1361)
Epoch 19/30 | Train Acc: 0.9147 | Train Loss: 0.3264 | Val Acc: 0.8816 | Val
Loss: 0.2813
  -> No improvement for 1 epoch(s).
Epoch 20/30 | Train Acc: 0.9136 | Train Loss: 0.3232 | Val Acc: 0.9377 | Val
Loss: 0.2149
  -> No improvement for 2 epoch(s).
Epoch 21/30 | Train Acc: 0.9136 | Train Loss: 0.3142 | Val Acc: 0.9283 | Val
Loss: 0.2016
  -> No improvement for 3 epoch(s).
Epoch 22/30 | Train Acc: 0.9042 | Train Loss: 0.3338 | Val Acc: 0.9969 | Val
Loss: 0.0863
  -> Saved new best model! (Val Loss: 0.0863)
Epoch 23/30 | Train Acc: 0.9128 | Train Loss: 0.3041 | Val Acc: 0.9315 | Val
Loss: 0.1640
  -> No improvement for 1 epoch(s).
Epoch 24/30 | Train Acc: 0.9428 | Train Loss: 0.2264 | Val Acc: 0.9377 | Val
Loss: 0.1392
  -> No improvement for 2 epoch(s).
Epoch 25/30 | Train Acc: 0.9221 | Train Loss: 0.2823 | Val Acc: 0.8474 | Val
Loss: 0.3718
  -> No improvement for 3 epoch(s).
Epoch 26/30 | Train Acc: 0.9443 | Train Loss: 0.2206 | Val Acc: 0.8629 | Val
Loss: 0.4521
  -> No improvement for 4 epoch(s).
Epoch 27/30 | Train Acc: 0.9447 | Train Loss: 0.2108 | Val Acc: 0.9346 | Val
Loss: 0.1514
  -> No improvement for 5 epoch(s).
Epoch 28/30 | Train Acc: 0.9517 | Train Loss: 0.1912 | Val Acc: 0.9221 | Val
Loss: 0.2081
  -> No improvement for 6 epoch(s).
Epoch 29/30 | Train Acc: 0.9431 | Train Loss: 0.2105 | Val Acc: 0.9221 | Val
Loss: 0.1573
  -> No improvement for 7 epoch(s).

[!] Early stopping triggered! Validation loss hasn't improved for 7 epochs.
[!] Halting training to prevent overfitting.


Generating training history plots…
Saved training history plots as 'training_history_plots_cnn.png'.
```

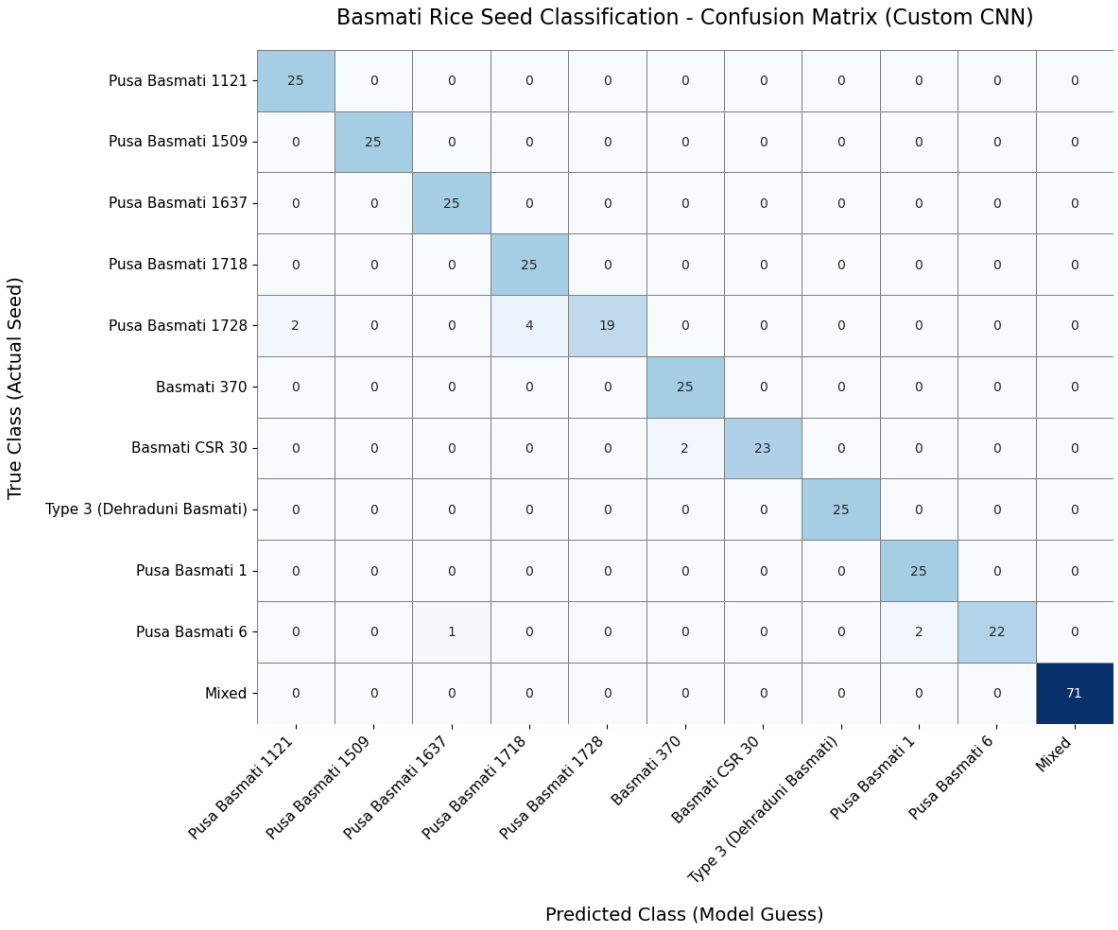


Evaluating best model on Test Set…

Classification Report:

	precision	recall	f1-score	support
Pusa Basmati 1121	0.93	1.00	0.96	25
Pusa Basmati 1509	1.00	1.00	1.00	25
Pusa Basmati 1637	0.96	1.00	0.98	25
Pusa Basmati 1718	0.86	1.00	0.93	25
Pusa Basmati 1728	1.00	0.76	0.86	25
Basmati 370	0.93	1.00	0.96	25
Basmati CSR 30	1.00	0.92	0.96	25
Type 3 (Dehraduni Basmati)	1.00	1.00	1.00	25
Pusa Basmati 1	0.93	1.00	0.96	25
Pusa Basmati 6	1.00	0.88	0.94	25
Mixed	1.00	1.00	1.00	71
accuracy			0.97	321
macro avg	0.96	0.96	0.96	321
weighted avg	0.97	0.97	0.96	321

Saved confusion matrix heatmap as 'confusion_matrix_heatmap_cnn.png'.


Basmati Rice Seed Classification - Confusion Matrix (Custom CNN)
True Class (Actual Seed)
Predicted Class (Model Guess)
Pusa Basmati 1121
Pusa Basmati 1509
Pusa Basmati 1637
Pusa Basmati 1718
Pusa Basmati 1728
Basmati 370
Basmati CSR 30
Type 3 (Dehraduni Basmati)
Pusa Basmati 1
Pusa Basmati 6
Mixed
25 0 0 0 0 0 0 0 0 0 0
0 25 0 0 0 0 0 0 0 0 0
0 0 25 0 0 0 0 0 0 0 0
0 0 0 25 0 0 0 0 0 0 0
2 0 0 4 19 0 0 0 0 0 0
0 0 0 0 0 25 0 0 0 0 0
0 0 0 0 0 2 23 0 0 0 0
0 0 0 0 0 0 0 25 0 0 0
0 0 0 0 0 0 0 0 25 0 0
0 0 1 0 0 0 0 0 2 22 0
0 0 0 0 0 0 0 0 0 0 71

[ ]: